

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples

- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

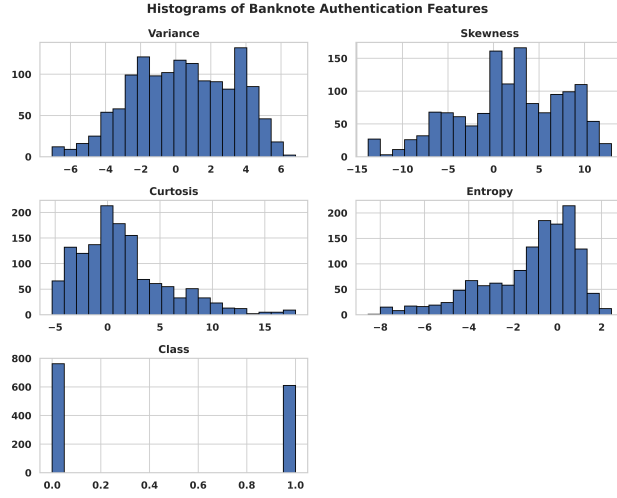


Figure 1: Histogram

the histogram shows the distribution of the values of the particular features in the dataset which are the following: variance, skewness, kurtosis, entropy. variance and skewness have the distribution at a larger range, meanwhile kurtosis and entropy have asymmetric and concentrated values around a smaller range

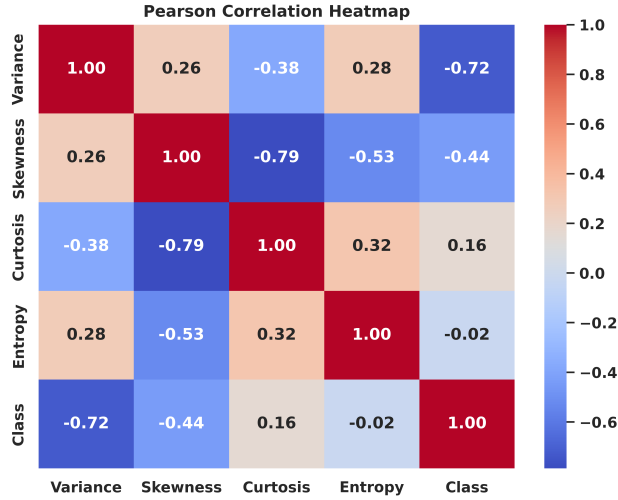


Figure 2: correlation heatmap

the correlation heatmap shows the linear relationships among the features. from the heatmap we can understand that skewness and kurtosis has the strongest feature correlation. this shows that banknotes with high skewness value has a low kurtosis value. Overall, the heatmap indicates that the dataset is absent of redundant feature, which aids in achieving high classification accuracy for the perceptron

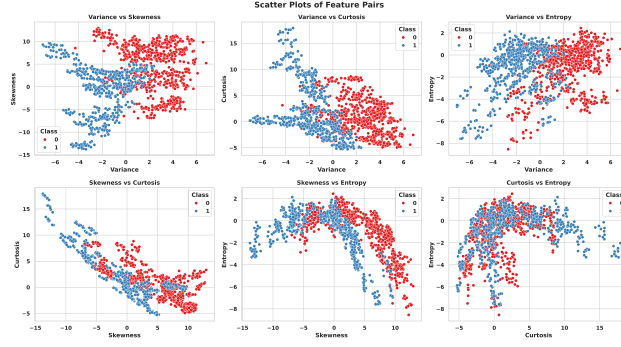


Figure 3: scatter plot

the scatter plots depicts the pairwise relationship between the 4 features where the pair of features distinguish the 2 class labels of genuine and forged notes. while most scatter plots have slight overlap between the classes, a few scatter plots have been to form distinguish groups. the maximum separation between the two classes have been done by the pair: skewness and variance

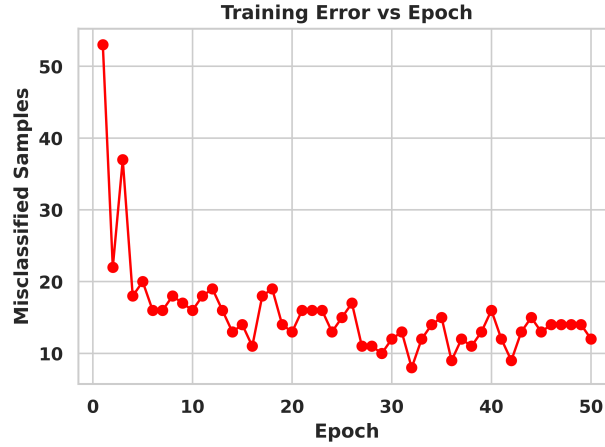


Figure 4: training error vs epoch

the graph tells us about the number of misclassified samples in each training epoch. In the initial training epoch, we notice high number of misclassified sample as the perceptron trying to learn, as the training reaches 50 epoch, the error gradually decreases and stabilize. This also shows that the learning rate chosen was sufficient for learning linearly separable hyperplane for the perceptron algorithm

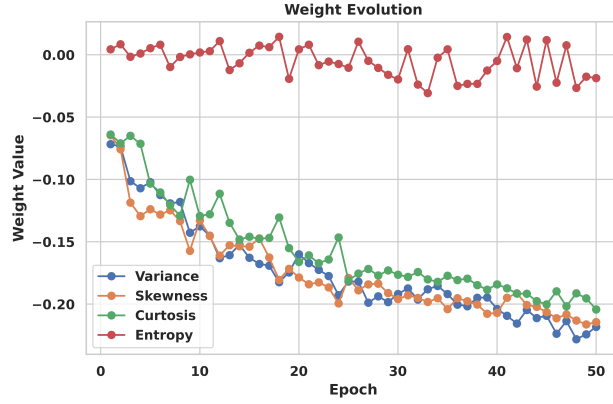


Figure 5: weight evolution

the figure shows the feature weights changing as the perceptron is learning to understand the importance of each feature. In the initial epoch, we notice rapid changes and the updates becoming smaller with each epoch, indicating convergence of the model and the weight values stabilizing and representing their relative importance in classification

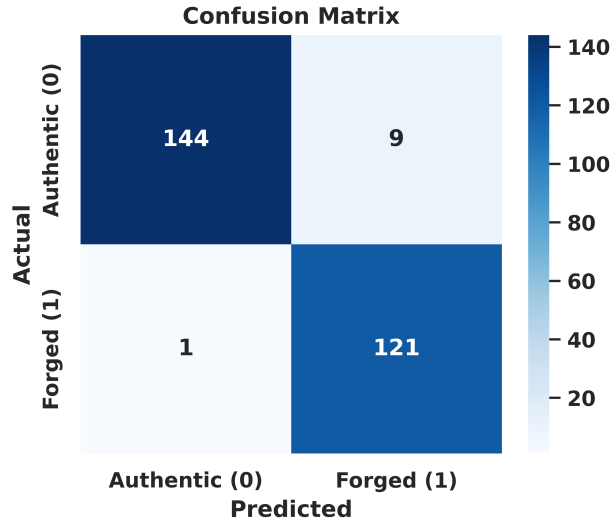


Figure 6: confusion matrix

the confusion matrix depicts high no of true negative and true positives, which shows good classification and identifying all forged and genuine notes correctly. this shows perceptron is able to classify the notes successfully

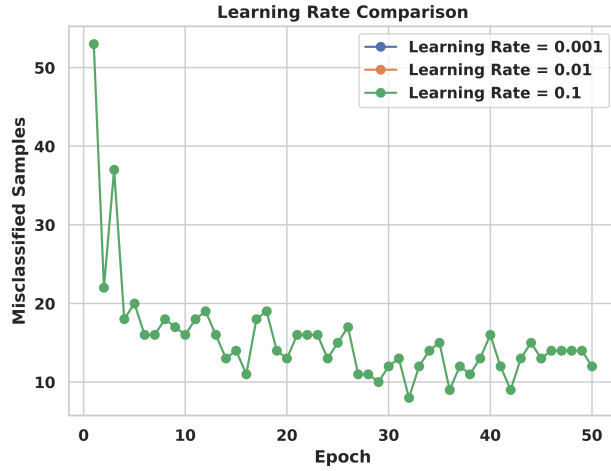


Figure 7: learning rate comparison

the graph shows that the 3 learning rates have identical convergence because of the feature values becoming normalized, and the perceptron had reached a similar decision boundary irrespective of the change in the learning rate. the learning rate mostly changed the weight magnitude updates rather than classification performance

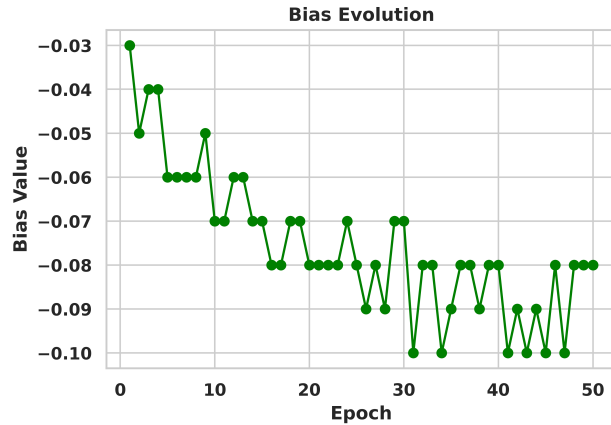


Figure 8: bias evolution

Initially, the bias undergoes a rapid change because the perceptron is still learning and attempts to shift the decision boundary for improvement in classification. Around epoch it starts to converge around -0.08. this tells us that the perceptron has been to learn the minimum threshold required for the classification

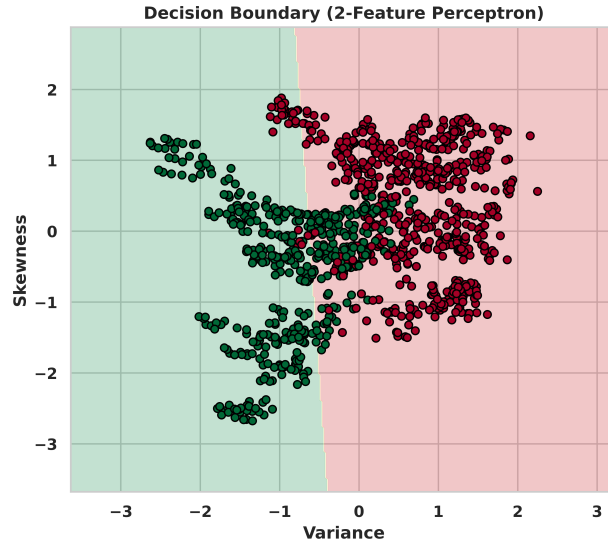


Figure 9: decision boundary

the following decision boundary visualized depicts that most of the samples have been classified correctly with only a few overlapping the other region. this shows that the dataset is linearly separable and the perceptron has been able to learn the linear boundary (hyperplane).

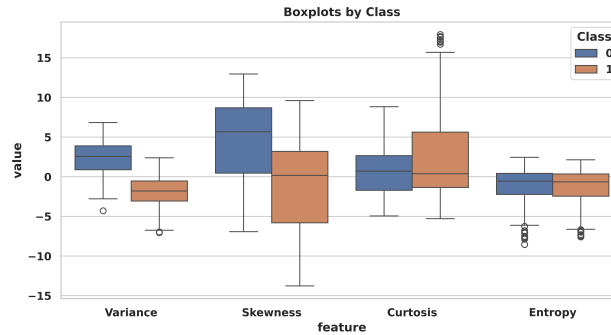


Figure 10: box plots by class

from the following graph, we can see that variance able to clearly distinguish between class 0 and class 1 (genuine and forged bank notes) while other features have slight overlap between the classes. The maximum spread of the classes is shown by skewness, indicating greater variability in the dataset

8. Performance Tables

Training Summary

Dataset Size	1372,5
Train/Test Split	80, 20
Learning Rate	0.01
Epochs	50
Final Weights	[-0.2182 -0.2145 -0.2043 -0.0187]
Final Bias	-0.08
Accuracy	0.9636
Precision	0.9308
Recall	0.9918
F1-score	0.9603

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
10	16	-0.1376	-0.1329	-0.07
20	13	-0.1600	-0.1786	-0.08
30	12	-0.1919	-0.1960	-0.07
40	16	-0.2039	-0.2071	-0.08
50	12	-0.2182	-0.2145	-0.08

9. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.
3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

RESULTS:

1. comparison between the two activation function

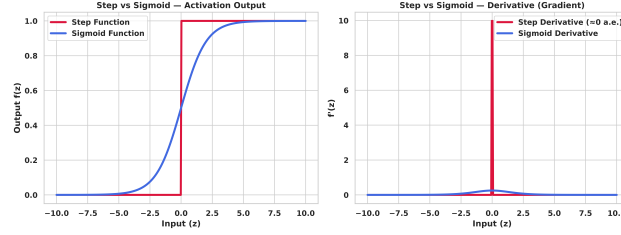


Figure 11: Step vs sigmoid function

While the step function classifies a particular sample in either class 0 or 1 based on a threshold learned by the perceptron algorithm, the sigmoid activation function is a continuous smooth curve. Even though the step function is computationally efficient, the function is not differentiable throughout the graph which will not be able to provide meaningful gradients like the sigmoid activation function

2. comparison between scikit-learn and scratch implementation of perceptron algorithm

Table 2: Performance Comparison: Scratch Perceptron vs Scikit-learn Perceptron

Metric	Scratch Perceptron	Scikit-learn Perceptron
Accuracy	0.9636	0.9455
Precision	0.9308	0.9280
Recall	0.9918	0.9508
F1-score	0.9603	0.9393

Table 3: Comparison of Learned Weights

Feature	Scratch Weight	Scikit-learn Weight
Variance	-0.2182	-0.1289
Skewness	-0.2145	-0.1303
Curtosis	-0.2043	-0.0837
Entropy	-0.0187	-0.0094
Bias	-0.0800	-0.0300

From the following data, both the models produce similar results, and comparable classification performance and decision boundary, the difference in values is only because the scikit learn model uses multiple optimization techniques and thus having a better computational efficiency

3. performance of the perceptron algorithm based on different learning rates

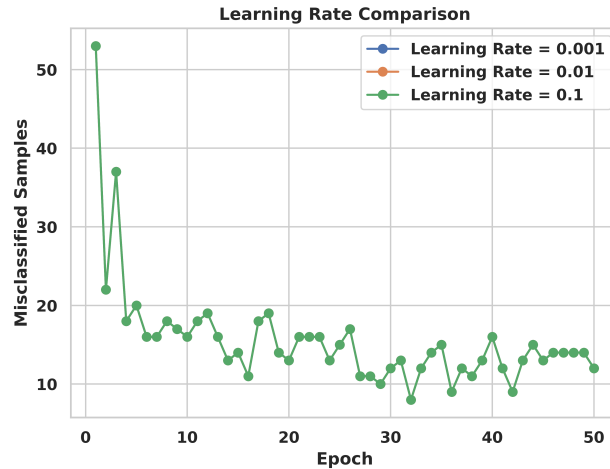


Figure 12: learning rate comparison

the graph shows that the 3 learning rates have identical convergence because of the feature values becoming normalized, and the perceptron had reached a similar decision boundary irrespective of the change in the learning rate. the learning rate mostly changed the weight magnitude updates rather than classification performance



Figure 13: XOR error vs epoch

The main reason why a single layer perceptron cannot solve XOR problem as it can only learn linear classification and decision boundary. Geometrically, the class 0 and class 1 points lie diagonally, hence creating a non-linear decision boundary as no single line can create a decision boundary to separate them

Table 4: Comparison of Feature Statistics Before and After Standardization

Feature	Raw Min	Raw Max	Raw Std.	Std. Min	Std. Max	Std. Std.
Variance	-7.04	6.82	2.84	-2.63	2.25	1.00
Skewness	-13.77	12.95	5.87	-2.68	1.88	1.00
Curtosis	-5.29	17.93	4.31	-1.55	3.84	1.00
Entropy	-8.55	2.45	2.10	-3.50	1.73	1.00

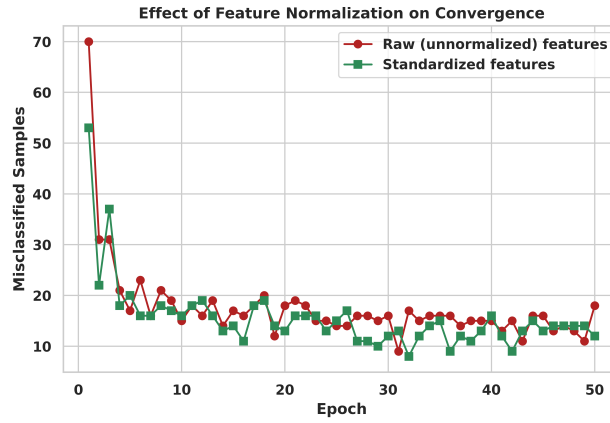


Figure 14: normalization convergence comparison

Feature normalization ensures that all input features are in a similar range to avoid dominance of a particular feature due to higher magnitude, which causes lower stability and slower convergence. After normalization, since all features are in a similar, each feature contributes equally in perceptron learning, thus resulting in smoother learning and faster convergence

10. Additional task - 23.07.2026

1. Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages. (K4)

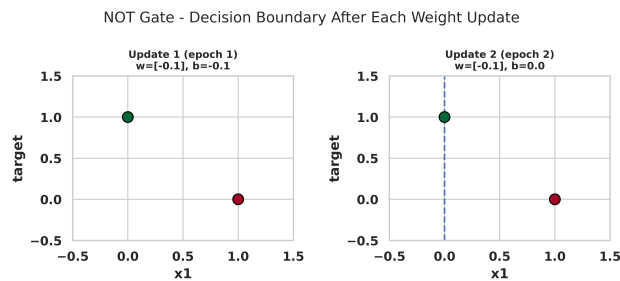


Figure 15: not boundary evolution

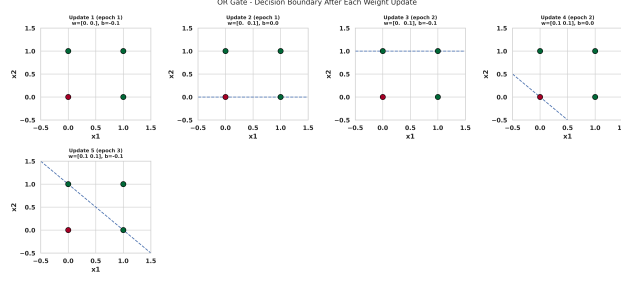


Figure 16: or boundary evolution

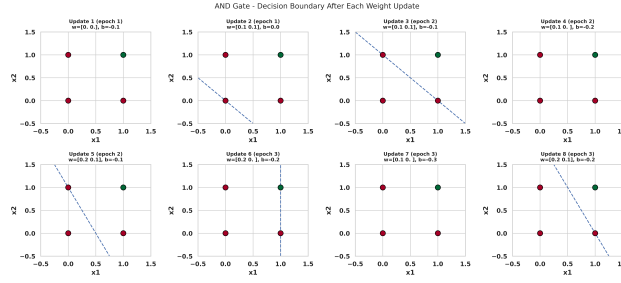


Figure 17: AND boundary evolution

The above 3 graphs depicting the decision boundary for the AND, OR and NOT logic gates helps us in understanding the fundamental working of the single layer perceptron. Initially, all the weights are randomly initialized and are updated through each epoch from learning from the misclassified samples. We can see that the OR and NOT logic gate decision boundary has converged faster than AND logic gate as it has a simpler class distribution.

The AND gate requires a longer time for convergence because of its positive class placement. But in the end, all the weights stabilize, indicating the convergence of the perceptron learning algorithm and was able to correctly learn to linearly separate them.

The above 3 pictures above show the process of how the decision boundary changes iteratively as it learns from the misclassified samples.

11. Discussions

The objective of the experiment was to implement a Single Layer Perceptron and evaluate its performance based on the banknote authentication dataset. The model performance was well which implied that it could linearly separate the data for classification.

12. Conclusion

The model was able to perform well for the given dataset. From the confusion matrix, we can see that only a few number of samples have been misclassified. Overall, the model was able to generalize well to be able to distinctly classify between genuine and forged bank notes. As the single layer perceptron is a primary and one of the first neural networks, it has its limitations.

not being to classify non-linearly separable dataset and the oscillation of convergence for the XOR problem

13. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code
- Experimental Results
- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

14. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.

LAB 1- github link